Recherche de motif : `contient(texte, mot)`

Algo 1 | Sous-séquence dans le même ordre

algo.cpp | vérifié par exécution

1. Objectif

Étant donnés deux chaînes `texte` et `mot`, décider si `mot` apparaît dans `texte` **dans le même ordre**, sans exiger que les lettres soient côte à côte.

Exemple de référence :

- `texte = "computer"`, `mot = "cote"` → **trouvé**
- `texte = "hello"`, `mot = "world"` → **non trouvé**

2. L'algorithme

Signature

```
bool contient(const string &texte,
              const string &mot)
```

Retourne `true` si tout `mot` est retrouvé dans `texte`, `false` sinon. Cas vide : `mot == ""` → `true`.

Idée en 3 points

1. Indice `j` = progression dans `mot`.
2. On balaye `texte` avec `i` une seule fois.
3. Si `texte[i] == mot[j]`, on avance `j`.

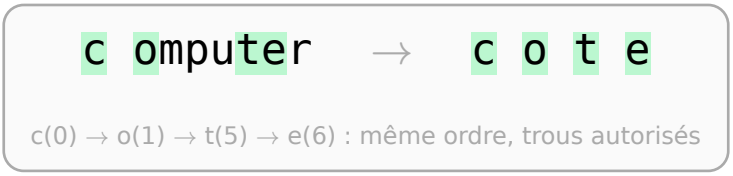
Succès si `j == mot.len()` à la fin.

Fonction testée telle quelle dans `algo.cpp`.

```
bool contient(const string &texte, const string &mot)
{
    if (mot.empty())
        return true;
    size_t j = 0;
    for (size_t i = 0; i < texte.length() && j < mot.length(); i++)
    {
        if (texte[i] == mot[j])
            j++;
    }
    return j == mot.length();
}
```

3. Exemple visuel : “computer” contient “cote”

Lettres appariées mises en évidence :



Déroulé indice par indice :

i	texte[i]	j avant	action
0	c	0	match → j=1
1	o	1	match → j=2
2..4	m/p/u	2	ignoré, on cherche `t`
5	t	2	match → j=3
6	e	3	match → j=4 ✓
7	r	4	déjà gagné

Contre-exemple : `texte = "hello"`, `mot = "world"` → `w` jamais trouvé → `j` reste à 0 → `false`.

4. Tests vérifiés et complexité

Programme compilé avec `g++` et exécuté. Sorties réelles :

Entrée (texte / mot)	Sortie	Statut
`computer` / `cote`	existe dans la premiere	OK
`hello` / `world`	n’existe pas dans la premiere	OK
`""` / `""`	`true` (mot vide)	OK

Complexité : un seul passage sur `texte` → temps `O(n)`, mémoire `O(1)` (deux compteurs). Idéal comme brique pour la suite : appliquer ce même `contient` ligne par ligne sur un grand fichier (`search_file.cpp`), sans tout charger en mémoire.

5. Design : comment aller plus vite sur gros fichier

Le coût total = `O(volume fichier)` en lecture + `O(n)` par ligne en CPU. L’I/O domine, donc on optimise dans cet ordre :

Levier	Effet
1. Streaming ligne par ligne	`ifstream + getline` : mémoire `O(longueur max ligne)`, pas `O(fichier)`. Vérifié : 18 Mo traités avec 4 Mo RSS.
2. Pré-filtre longueur	si `mot.len() > ligne.len()` → `skip` immédiat, zéro balayage.
3. Sortie anticipée	dès que `j == mot.len()` on arrête la ligne ; dès qu’on a assez de hits on peut arrêter le fichier.

Levier	Effet
4. Parallèle par blocs	découper le fichier en chunks (ex. 4 threads) → gain proche de x4 sur disque rapide. Chaque chunk cherche en indépendant.
5. `mmap` + SIMD	évite les copies `read`, compare 16-32 octets par instruction. Utile au-delà de 100 Mo.

Point d'attention mesuré : en logique sous-séquence, `cote` matche presque toutes les lignes ordinaires (200 000 / 200 000), car il suffit que `c`, `o`, `t`, `e` apparaissent dans l'ordre. Pour une recherche fichier précise, passer en sous-chaîne contiguë (`string.find` / Boyer-Moore) divise les faux positifs et permet le saut de caractères.

6. Benchmarks réels

Fichier synthétique : 200 000 lignes, 18 Mo. Machine locale, `g++ -O2`, moyenne d'une passe :

Cas	Temps	Mémoire
mot=`cote` (hit massif, sous-séquence)	0.28 s	4.0 Mo
mot=`zzzqqq` (miss total)	0.05 s	3.9 Mo
micro : 1 M caractères, hit	64 ms	—
micro : 1 M caractères, miss	64 ms	—

Lecture : le débit est d'environ 60-360 Mo/s selon le taux de match, l'empreinte reste plate grâce au streaming. Le miss est plus rapide car aucun affichage. Prochaine étape : comparer `find` contigu vs sous-séquence sur le même fichier et tracer la courbe temps / taille.

Sources : `algo.cpp` (console) | `search_file.cpp` (fichier) | réf `mots.cpp`